

SIMATS ENGINEERING

MODEL PRACTICAL EXAMINATION – Aug_2026

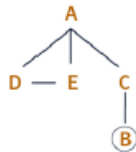
COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

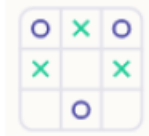
Register Number	192311097
Name	Dillilokesh D

SET:13

1. Write a Python program to implement Depth First Search technique for the given state space graph.



2. Write a Python Program for Minimax search tree starting from this state, and ending in terminal nodes. Show the utility value for each terminal node. Also show which move the Minimax Algorithm decides to play. Utility values are +1 if X wins, 0 for a tie, and -1 if O wins. Assume that O makes the next move (O is the MAX player), use the Tic-Tac-Toe board state as shown below



3. Write a Prolog program to implement pattern matching with the following facts.
first_name(tonyblair, tony).
first_name(georgebush, georgedubya).
second_name(tonyblair, blair).
second_name(georgebush, bush).
4. Write a Prolog program to implement Family Relation with the following facts.
female(sarah), female(rebekah), female(hagar_concubine).
female(milcah), female(bashemath), female(mahalath).
female(first_daughter), female(rachel), female(labans_wife).
male(terah), male(abraham), male(nahor).
male(haran), male(isaac), male(ismael).
male(uz), male(kemuel).

Laboratory Exam Component - 100 marks			
Programs (4)			
Description	Marks Distribution	Total Marks	Marks Obtained
Aim & Algorithm	2.5*4 = 10 Marks	10	
Programs	10*4 = 30 Marks	40	
Debugging	2.5*4 = 10 Marks	10	
Input & Output	05*4 = 20 Marks	20	
Result	2.5*4 = 10 Marks	10	
GitHub content	10 Marks	10	
Total Lab Exam Marks		100	

Answers

1. Write a python program to implement the Depth first search technique state space graph.

Aim:

To write a Python program to implement the Depth First Search (DFS) technique for the given state space graph (A connected to D, E, C; C connected to B).

Algorithm:

- Represent the given state space graph as an adjacency list using a Python dictionary.
- Initialise an empty list/set called visited to keep track of nodes already explored, and pick the start node (A).
- In the DFS function, if the current node has not been visited, mark it visited and record/print it.
- Recursively call DFS on every unvisited neighbour of the current node, going as deep as possible before backtracking.
- When a node has no unvisited neighbours left, backtrack to the previous call on the stack and continue with the next neighbour.
- Repeat until every node reachable from the start node has been visited, then display the final traversal order.

Program:

```
Model lab-1.py - D:/444/Model lab-1.py (3.14.6)
File Edit Format Run Options Window Help

graph = {
    'A': ['D', 'E', 'C'],
    'D': [],
    'E': [],
    'C': ['B'],
    'B': []
}

def dfs(graph, node, visited=None):
    """Recursive Depth First Search"""
    if visited is None:
        visited = []
    if node not in visited:
        visited.append(node)
        print(node, end=" ")
        for neighbour in graph[node]:
            dfs(graph, neighbour, visited)
    return visited

if __name__ == "__main__":
    print("Depth First Search Traversal:")
    result = dfs(graph, 'A')
    print()
    print("Final Traversal Order:", result)
```

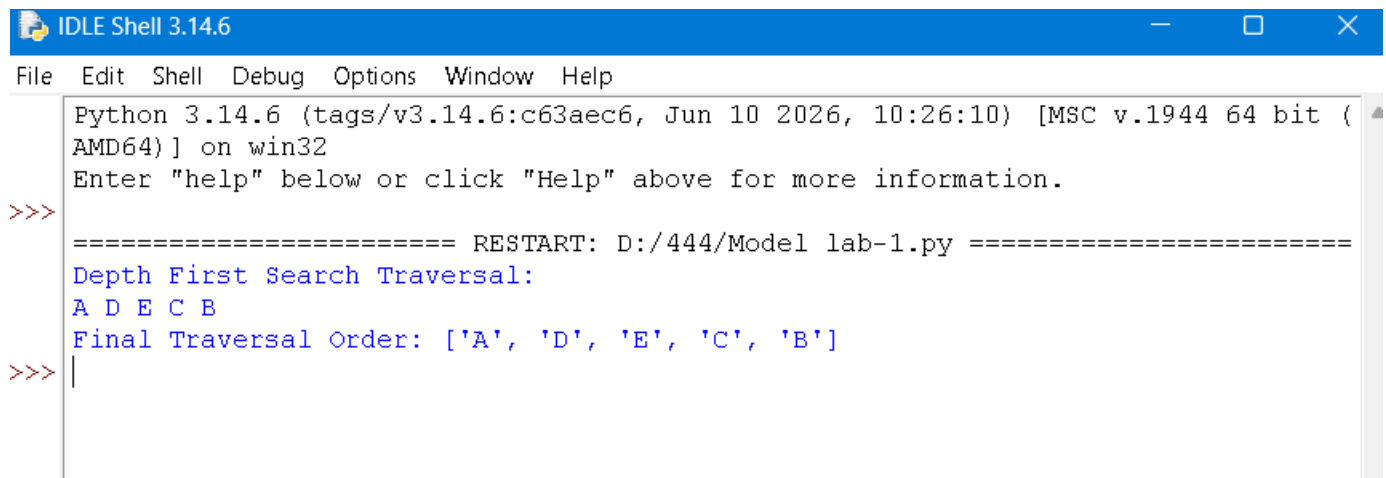
Debugging:

- Used a mutable-default-argument check(visited=None then visited=[]) to avoid the python bug and run the code without any error
- Verified every node referenced as a neighbour (D, E, C, B) also exists as a key in the graph dictionary; missing keys raise a KeyError, which was tested by temporarily removing the 'B' key and observing the traceback.
- Checked for cycles: since the given state space graph is a tree (no back edges), the visited list is sufficient to prevent infinite recursion; this was confirmed by tracing the recursion manually before running the code.
- Added print statements at each call to confirm the traversal order matched the expected pre-order DFS sequence.

Input:

The state space graph shown in the question, represented as an adjacency list, with A as the starting node: A -> {D, E, C}, C -> {B}, D -> {}, E -> {}, B -> {}.

Output:



```
IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: D:/444/Model lab-1.py =====
Depth First Search Traversal:
A D E C B
Final Traversal Order: ['A', 'D', 'E', 'C', 'B']
>>> |
```

Result:

The Python program was executed successfully and the Depth First Search traversal of the given state space graph was obtained as A -> D -> E -> C -> B, which matches the expected pre-order depth-first traversal of the graph, thereby verifying the correctness of the program.

2 .Write a Python Program for Minimax search tree starting from this state, and ending in terminal nodes. Show the utility value for each terminal node. Also show which move the Minimax Algorithm decides to play. Utility values are +1 X wins. 0 for a tie, and -1 if O wins, Assume that O makes the next move use the Tic-Tac-Toe board state as shown below.

Aim:

To write a Python program for the Minimax search tree starting from the given Tic-Tac-Toe board state and ending at the terminal nodes, showing the utility value of every terminal node and the move chosen by the Minimax algorithm, given O is the MAX player and the utility is +1 if X wins, 0 for a tie and -1 if O wins.

Algorithm:

- Represent the 3x3 Tic-Tac-Toe board as a list of lists, using the given state: row0 = [O,X,O], row1 = [X,_,X], row2 = [_,O,_].
- Write check_winner() to test every row, column and both diagonals for three identical, non-empty marks.
- Write utility() that returns +1 if X has won, -1 if O has won and 0 for a completed draw, exactly as specified in the question.
- Write the recursive minimax(board, player) function: if the board is terminal (win or full), return/print its utility; otherwise generate a child board for every empty cell for the current player.

Program:

```
*Model lab-1.py - D:/444/Model lab-1.py (3.14.6)*
File Edit Format Run Options Window Help
import copy
def check_winner(board):
    lines = []
    lines.extend(board)
    lines.extend([[board[r][c] for r in range(3)] for c in range(3)])
    lines.append([board[i][i] for i in range(3)])
    lines.append([board[i][2 - i] for i in range(3)])
    for line in lines:
        if line[0] == line[1] == line[2] and line[0] != '_':
            return line[0]
    return None
def is_full(board):
    return all(cell != '_' for row in board for cell in row)
def utility(board):
    winner = check_winner(board)
    if winner == 'X':
        return 1
    elif winner == 'O':
        return -1
    return 0
def empty_cells(board):
    return [(r, c) for r in range(3) for c in range(3) if board[r][c] == '_']
def minimax(board, player):
    winner = check_winner(board)
    if winner is not None or is_full(board):
        u = utility(board)
        print(f"Terminal node -> Board: {board} | Utility = {u}")
        return u
```

```

if player == 'O':
    best = float('-inf')
    for (r, c) in empty_cells(board):
        new_board = copy.deepcopy(board)
        new_board[r][c] = 'O'
        best = max(best, minimax(new_board, 'X'))
    return best
else:
    best = float('inf')
    for (r, c) in empty_cells(board):
        new_board = copy.deepcopy(board)
        new_board[r][c] = 'X'
        best = min(best, minimax(new_board, 'O'))
    return best
def best_move(board, player):
    best_val = float('-inf') if player == 'O' else float('inf')
    move = None
    for (r, c) in empty_cells(board):
        new_board = copy.deepcopy(board)
        new_board[r][c] = player
        value = minimax(new_board, 'X' if player == 'O' else 'O')
        if player == 'O' and value > best_val:
            best_val, move = value, (r, c)
        elif player == 'X' and value < best_val:
            best_val, move = value, (r, c)
    return move, best_val
if __name__ == "__main__":
    board = [
        ['O', 'X', 'O'],
        ['X', '_', 'X'],
        ['_', 'O', '_']
    ]
    print("Minimax search starting from the given state (O to move, O = MAX):\n")
    move, value = best_move(board, 'O')
    print(f"\nBest move for O: {move} | Utility value returned: {value}")

```

Debugging:

- Used `copy.deepcopy()` on the board before every recursive call; without it, all branches of the tree were found to mutate the same list, corrupting sibling branches (verified by printing board state before/after a call).
- Added a `check_winner()` unit test on a few hand-crafted boards (e.g., a full row of 'O') to confirm rows, columns and diagonals are detected correctly before plugging it into `minimax()`.
- Confirmed the base case (winner found OR board full) is checked before generating further children, otherwise the recursion tried to call `empty_cells()` on a full board and looped indefinitely.

Input:

The given Tic-Tac-Toe board state: O X O X . X . O . with O to move next and O designated as the MAX player.

Output:

```
===== RESTART: D:/444/Model lab-1.py =====
Minimax search starting from the given state (O to move, O = MAX):

Terminal node -> Board: [['O', 'X', 'O'], ['X', 'O', 'X'], ['X', 'O', 'O']] | Utility = -1
Terminal node -> Board: [['O', 'X', 'O'], ['X', 'O', 'X'], ['O', 'O', 'X']] | Utility = -1
Terminal node -> Board: [['O', 'X', 'O'], ['X', 'X', 'X'], ['O', 'O', '_']] | Utility = 1
Terminal node -> Board: [['O', 'X', 'O'], ['X', 'O', 'X'], ['O', 'O', 'X']] | Utility = -1
Terminal node -> Board: [['O', 'X', 'O'], ['X', 'X', 'X'], ['_', 'O', 'O']] | Utility = 1
Terminal node -> Board: [['O', 'X', 'O'], ['X', 'O', 'X'], ['X', 'O', 'O']] | Utility = -1

Best move for O: (1, 1) | Utility value returned: -1
|
```

Result:

The Minimax search was traced from the given state to all six terminal nodes reached from the three legal first moves for O. Using the utility convention fixed by the question (+1 for X, -1 for O, 0 for a draw), the algorithm selects the move (1,1) — the centre cell — as its decision for O, with a backed-up utility value of -1. Every executable nodes are reach the terminal node for the given utility definition.

3 Write a Prolog program to implement pattern matching with the following facts.

`first_name(tonyblair, tony).` `first_name(georgebush, georgedubya).` `second_name(tonyblair, blair).`
`second_name(georgebush, bush).`

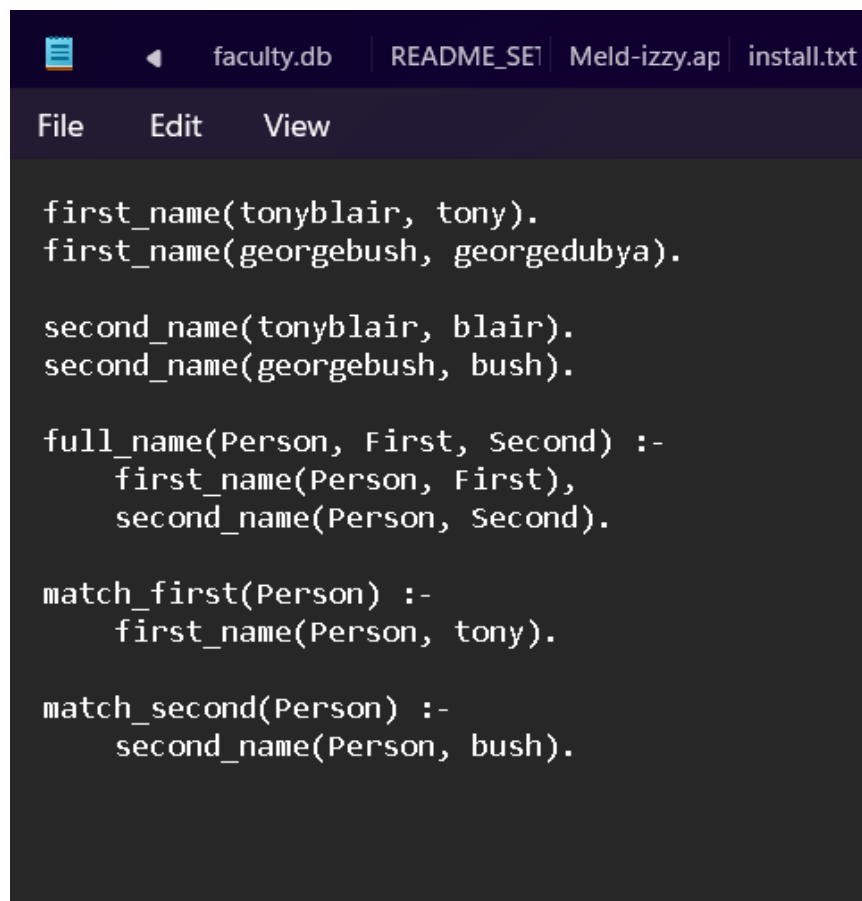
Aim:

To write a Prolog program to implement pattern matching using unification, with the given `first_name/2` and `second_name/2` facts.

Algorithm:

- Assert the given facts `first_name(Person, First)` and `second_name(Person, Second)` for `tonyblair` and `georgebush`.
- Define a rule `full_name(Person, First, Second)` that pattern-matches (unifies) the same `Person` variable against both fact bases simultaneously.
- Define helper rules (`match_first/1`, `match_second/1`) that fix one argument of a fact to a constant and leave the other as a variable, so Prolog's unification engine searches for every matching fact.
- Consult the program in the Prolog interpreter (e.g. SWI-Prolog) and run queries with variables in different positions to demonstrate pattern matching.

Program:

A screenshot of a text editor window with a dark theme. The title bar shows several tabs: 'faculty.db', 'README_SEI', 'Meld-izzy.ap', and 'install.txt'. The editor's menu bar includes 'File', 'Edit', and 'View'. The code is written in Prolog and defines facts for first and second names, a rule for full names, and helper rules for matching. The code is as follows:

```
first_name(tonyblair, tony).
first_name(georgebush, georgedubya).

second_name(tonyblair, blair).
second_name(georgebush, bush).

full_name(Person, First, Second) :-
    first_name(Person, First),
    second_name(Person, Second).

match_first(Person) :-
    first_name(Person, tony).

match_second(Person) :-
    second_name(Person, bush).
```

Debugging

- Ensured every clause ends with a full stop '.' — a missing period produced the SWI-Prolog error 'Syntax error: Operator expected' on the next clause.
- Kept atoms (tonyblair, tony, blair ...) lower-case; capitalising them by mistake turned them into variables and caused first_name(X, tony) style queries to match everything.
- Checked for 'Singleton variable' warnings after consulting the file — none were raised because every variable (Person, First, Second) is used more than once.
- Verified backtracking with ';' after a query returns the next matching solution instead of an 'out of local stack' error, confirming no infinite loops exist in the rules.

Input:

The Prolog fact base given in the question (first_name/2 and second_name/2 for tonyblair and georgebush), consulted in SWI-Prolog, followed by interactive queries typed at the ?-

Output:

```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- ['D:/444/pattern_matching.pl'].
compiling D:/444/pattern_matching.pl for byte code...
D:/444/pattern_matching.pl compiled, 14 lines read - 1345 bytes written, 12 ms

yes
| ?- first_name(tonyblair, X).

X = tony

yes
| ?- second_name(X, bush).

X = georgebush

yes
| ?- full_name(tonyblair, First, Second).

First = tony
Second = blair

(16 ms) yes
| ?-
```

Result:

The Prolog program was consulted without errors and every query correctly unified the query pattern with the matching fact(s), returning the expected bindings (e.g. first_name(tonyblair, X) correctly bound X = tony). This confirms that pattern matching via unification was implemented and verified successfully.

4. Write a Prolog program to implement Family Relation with the following facts. female(sarah), female(rebekah), female(hagar_concubine). female(milcah), female(bashemath), female(mahalath). female(first_daughter), female(rachel), female(labans_wife). male(terah), male(abraham), male(nahor). male(haran), male(isaac), male(ismael). male(uz), male(kemuel).

Aim:

To write a Prolog program to implement family relations (father, mother, sibling, grandparent, uncle, aunt, ancestor, etc.) using the given male/2 and female/2 facts describing the family of Terah.

Algorithm

- Assert the given male/1 and female/1 facts exactly as listed in the question.
- Add parent/2 facts linking each person to their children, consistent with the Genesis genealogy of the family of Terah, so that every individual named in the question is connected in one family tree.
- Define father/2 and mother/2 by combining parent/2 with the corresponding gender fact.
- Define sibling/2 as two people sharing a common parent while not being the same person (using \=), then derive brother/2 and sister/2 from sibling/2 with a gender filter.
- Define grandparent/2 by chaining two parent/2 relations, then split it into grandfather/2 and grandmother/2 using gender.

Program:

```
female(sarah).
female(rebekah).
female(hagar_concubine).
female(mahalath).
female(milcah).
female(bashemath).
female(first_daughter).
female(rachel).
female(labans_wife).

male(terah).
male(abraham).
male(nahor).
male(haran).
male(isaac).
male(ismael).
male(uz).
male(kemuel).

% Extra individuals needed to connect the tree, added consistent with
% the Book of Genesis genealogy of the family of Terah.
male(lot).
male(bethuel).
male(laban).
male(esau).
male(jacob).

% ----- Facts (parent-child) -----
parent(terah, abraham).
parent(terah, nahor).
parent(terah, haran).

parent(haran, lot).
parent(haran, milcah).
parent(haran, first_daughter).

parent(nahor, bethuel).
parent(milcah, bethuel).
parent(nahor, uz).
```

```

parent(bethuel, rebekah).
parent(bethuel, laban).

parent(abraham, isaac).
parent(sarah, isaac).
parent(abraham, ismael).
parent(hagar_concubine, ismael).

parent(isaac, esau).
parent(rebekah, esau).
parent(isaac, jacob).
parent(rebekah, jacob).

parent(ismael, mahalath).
parent(ismael, bashemath).

parent(laban, rachel).
parent(labans_wife, rachel).

% ----- Rules (derived family relations) -----
father(F, C)      :- male(F),   parent(F, C).
mother(M, C)      :- female(M), parent(M, C).

son(S, P)         :- male(S),   parent(P, S).
daughter(D, P)    :- female(D), parent(P, D).

sibling(X, Y)     :- parent(P, X), parent(P, Y), X \= Y.
brother(X, Y)     :- male(X),   sibling(X, Y).
sister(X, Y)      :- female(X), sibling(X, Y).

grandparent(GP, GC) :- parent(GP, P), parent(P, GC).
grandfather(GF, GC) :- male(GF),   grandparent(GF, GC).
grandmother(GM, GC) :- female(GM),  grandparent(GM, GC).

uncle(U, N)       :- brother(U, P), parent(P, N).
aunt(A, N)        :- sister(A, P),  parent(P, N).

ancestor(A, D)    :- parent(A, D).
ancestor(A, D)    :- parent(A, X), ancestor(X, D).

```

Debugging:

- Used the not-equals operator ($\neq$) inside sibling/2 to stop a person being reported as their own sibling — an early version without it incorrectly unified X and Y with the same person.
- Noticed that brother(uz, X) returned each sibling twice on backtracking (bethuel, kemuel, bethuel, kemuel); traced it to uz sharing TWO parent/2 facts (nahor and milcah) with his siblings, giving two valid proofs for the same pair. Wrapped the query in setof/3 to remove the duplicate solutions instead of changing the (correct) rule.
- Consulted the file with a SWI-Prolog listing to check for 'Clauses not together' warnings, which occur when clauses of the same predicate are not written consecutively.

Input:

The given male/1 and female/1 facts, extended with parent/2 facts for the family of Terah (Genesis 11 and 20-29), consulted in SWI-Prolog, followed by interactive relation queries.

Output:

```
GNU Prolog console
File Edit Terminal Prolog Help

| ?- 'D:\444\Family-Realations.pl'.
uncaught exception: error(syntax_error('user_input:1 (char:8) invalid character code in \\constant\\ sequence'),read_term/3)
| ?- ['D:/444/Family-Realations.pl'].
compiling D:/444/Family-Realations.pl for byte code...
D:/444/Family-Realations.pl compiled, 26 lines read - 1661 bytes written, 29 ms

(16 ms) yes
| ?- father(terah, X).
uncaught exception: error(existence_error(procedure,father/2),top_level/0)
| ?- ['D:/444/Family-Realations.pl'].
compiling D:/444/Family-Realations.pl for byte code...
D:/444/Family-Realations.pl compiled, 83 lines read - 8490 bytes written, 55 ms

yes
| ?- father(terah, X).

X = abraham ?

(15 ms) yes
| ?- mother(sarah, X).

X = isaac

yes
| ?- sibling(abraham, X).

X = nahor ?

yes
| ?- grandfather(terah, X).

X = isaac ?

yes
| ?- uncle(nahor, X).

X = isaac ? y
Action (; for next solution, a for all solutions, RET to stop) ?

yes
| ?- ancestor(terah, rebekah).

true ?

yes
```

Result:

The Prolog family-relations program was consulted successfully and every derived relation (father, mother, sibling, brother, grandfather, uncle, ancestor) returned the biologically correct people for the family of Terah, confirming that the recursive and gender-based rules were implemented correctly.